

LearningSteps Evolution

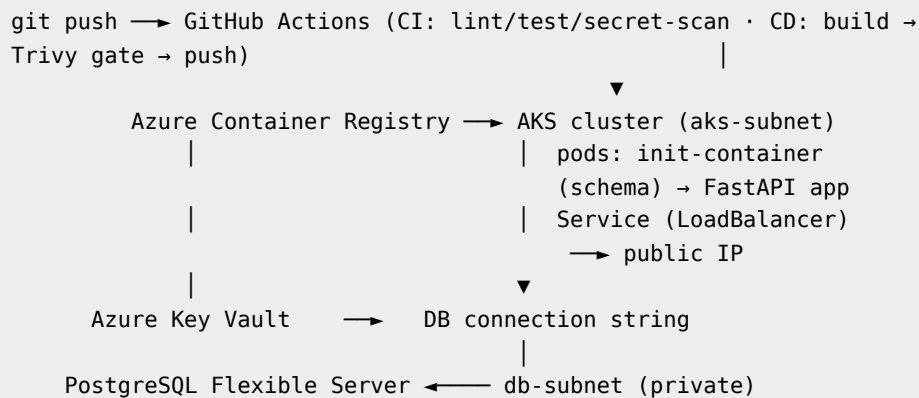
From manual to automated deployment - DevOps

Training Program by: Cybersteps

Cohort: 12/25

Name: E-Itohan

Date: 04.10.2026

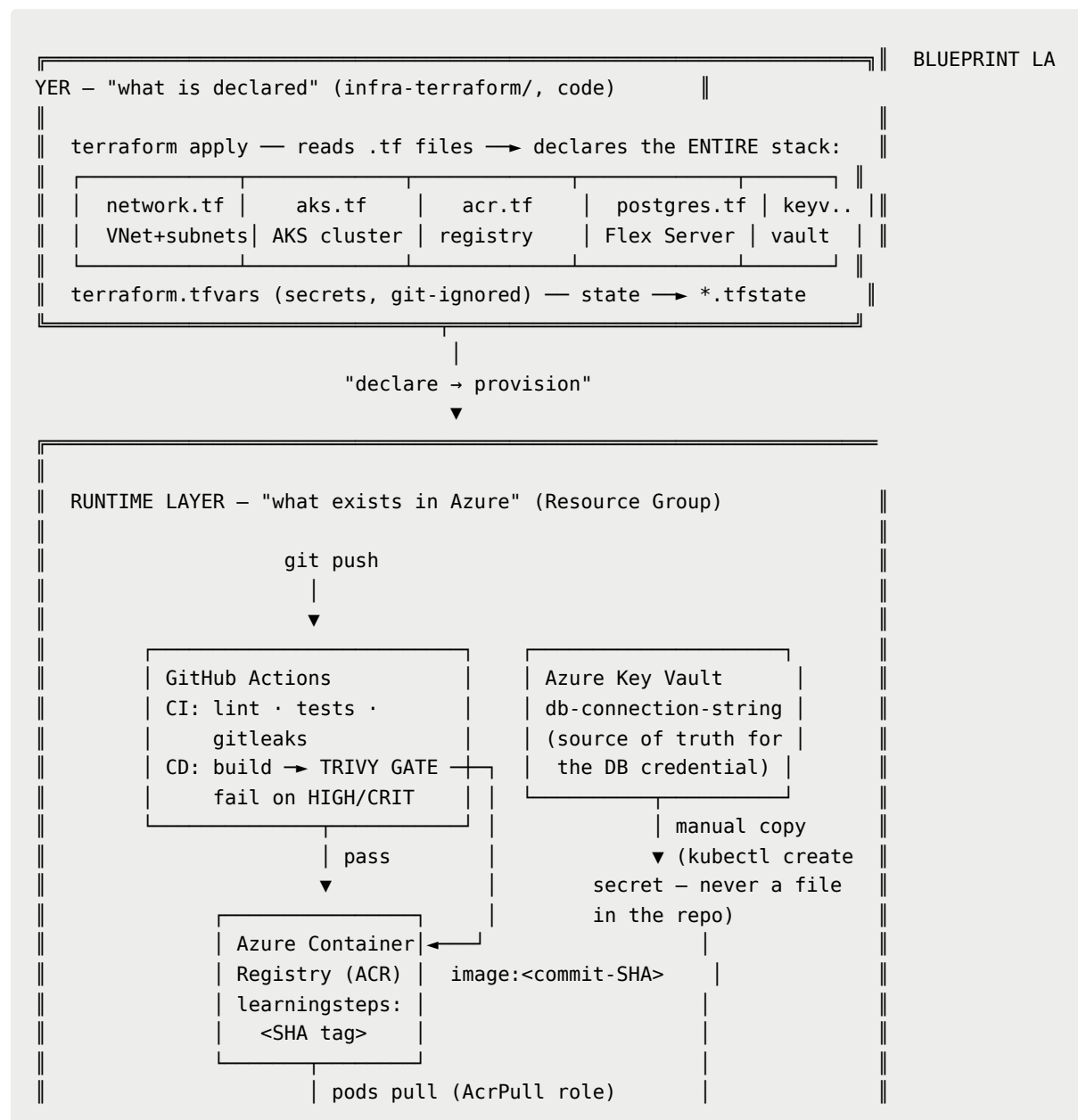


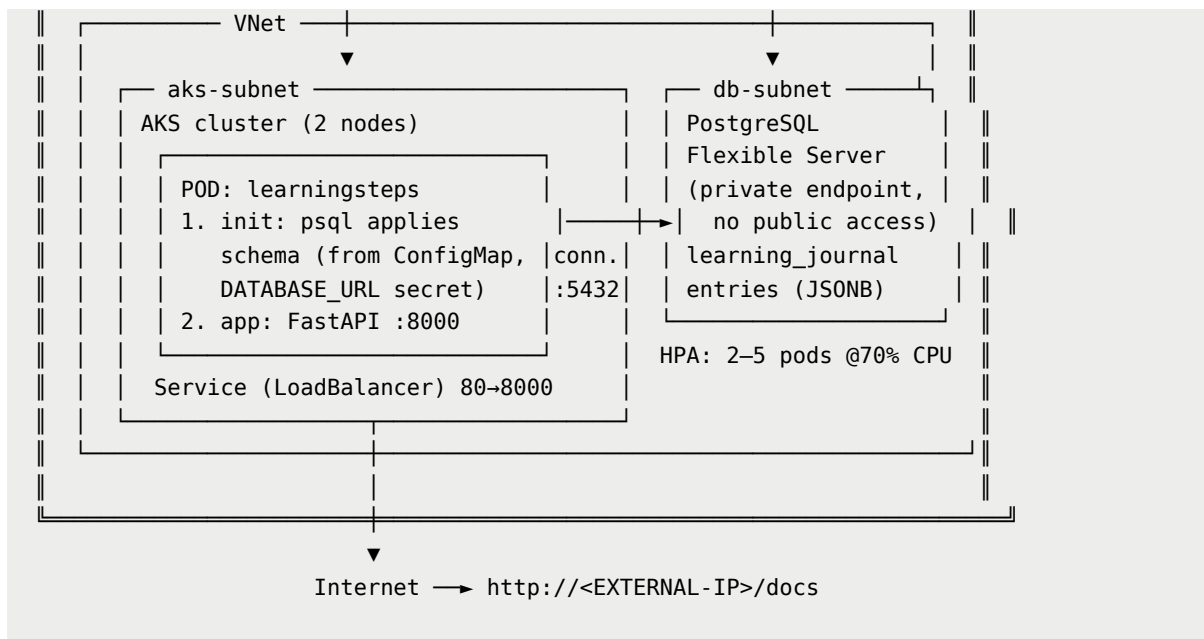
1. Objective

Take the LearningSteps journal API (FastAPI + PostgreSQL), previously deployed as two hand-configured Azure VMs in the *Origins* project, and wrap it in a full DevOps lifecycle: containerization (Docker), infrastructure as code (Terraform), orchestration (Azure Kubernetes Service), and automated delivery with integrated security scanning (GitHub Actions + Trivy). The guiding principle of the assignment: move from *clicking resources into existence* to *declaring them as reproducible code*.

2. Architecture

The system spans four layers:





- **Infrastructure** (Terraform): Resource Group, VNet with delegated subnets for AKS and PostgreSQL, ACR, AKS cluster (2×Standard_D2s_v3 nodes) with `AcrPull` role assignment, PostgreSQL Flexible Server, Key Vault holding the connection string.
- **Application** (Docker): `python:3.13-slim` base, pinned dependencies, uvicorn entrypoint.
- **Orchestration** (Kubernetes): Deployment (replicas: 2, resource requests for HPA compatibility) with an **init container** that applies the database schema (`postgres:15-alpine` running `database_setup.sql` via ConfigMap), a LoadBalancer Service (80 → 8000), an HPA (2-5 pods, 70% CPU), and ConfigMap + Secret separation for non-secret config vs. credentials.
- **Pipeline** (GitHub Actions): CI workflow (lint, tests, gitleaks secret scanning) and CD workflow (build → Trivy vulnerability scan → push to ACR, tagged by commit SHA).

3. Implementation

3.1 Containerization

The Dockerfile builds a slim Python 3.13 image, upgrades a tracked Debian package (see §5), installs pinned dependencies from `api/requirements.txt`, and runs uvicorn. Local verification was done via `docker-compose.yml` (API + PostgreSQL 16 + healthcheck-gated startup ordering). Dependency floors were added to eliminate known CVEs (§5.2).

```

FROM python:3.13-slim

RUN apt-get update && \
    apt-get upgrade -y --only-upgrade libpcre2-8-0 && \
    rm -rf /var/lib/apt/lists/*

WORKDIR /FastAPI

COPY api/requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY api/ .

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port",
    "8000"]

```

Dockerfile

3.2 Infrastructure as Code

All Azure resources are declared in Terraform. Key decisions and lessons:

- **Global uniqueness constraints:** the ACR name, Key Vault name, and PostgreSQL server name (which becomes a public DNS name) must be globally unique.
- **Explicit dependency ordering:** most ordering is derived automatically from resource references, but the private DNS-zone link must precede the PostgreSQL server, and the Key Vault role assignment must precede writing the secret.
- **State and secrets hygiene:** `terraform.tfvars` (subscription ID, DB password), `.terraform/`, and `.tfstate` are git-ignored; the lock file is committed ("commit the recipe, not the ingredients").
- **Region constraint:** PostgreSQL Flexible Server availability varies by region/subscription, verified with `az postgres flexible-server list-skus` before committing to the selected region.

3.3 DevSecOps Pipeline

Two workflows. CI: gitleaks full-history secret scanning, ruff lint, pytest, image build. CD: ACR login, image build tagged with the commit SHA, **Trivy scan configured as a quality gate** (`severity: HIGH,CRITICAL` , `exit-code: 1` , `ignore-unfixed: true`), push to ACR only on success. Findings suppressed explicitly and with justification in `.trivyignore` (see §5.3).

```

name: LearningSteps CI
on:
  push:
    branches: [ "main" ]
  pull_request:
  workflow_dispatch:

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v5

      - name: Set up Python
        uses: actions/setup-python@v6
        with:
          python-version: "3.13"

      - name: Install dependencies
        run: pip install -r api/requirements.txt

      - name: Lint
        run: |
          pip install ruff
          ruff check . || true

      - name: Run tests
        run: |
          pip install pytest
          pytest -q || echo "no tests yet - add them"

      - name: Build Docker image
        run: docker build -t learningsteps:${{ github.sha
}}} .

secret-scan:
  name: Secret Scan

```

```
runs-on: ubuntu-latest
steps:
  - name: Checkout full history
    uses: actions/checkout@v5
    with:
      fetch-depth: 0

  - name: Run gitleaks scan
    uses: gitleaks/gitleaks-action@v2
    with:
      github_token: ${{ secrets.GITHUB_TOKEN }}
```

CI code

```

name: LearningSteps CD

on:
  push:
    branches:
      - main
  workflow_dispatch:

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v5

      - name: Login to Azure Container Registry
        uses: docker/login-action@v3
        with:
          registry: ${ secrets.ACR_SERVER }
          username: ${ secrets.ACR_USERNAME }
          password: ${ secrets.ACR_PASSWORD }

      - name: Build Docker image
        run: |
          docker build -t ${ secrets.ACR_SERVER }/learningsteps:${ github.sha } .

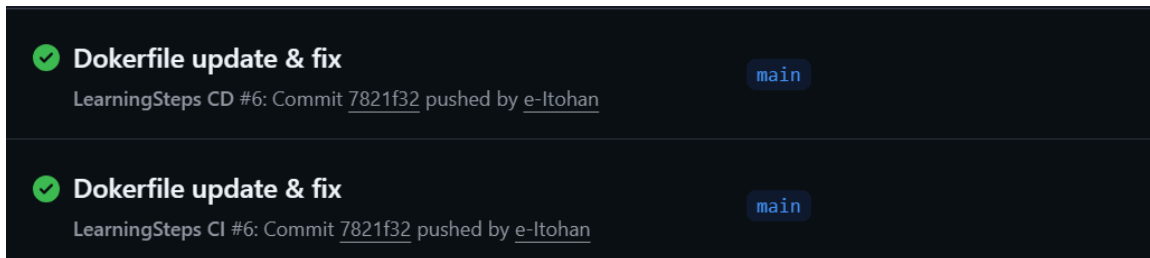
      - name: Scan image with Trivy
        uses: aquasecurity/trivy-action@v0.36.0
        with:
          image-ref: ${ secrets.ACR_SERVER }/learningsteps:${ github.sha }
          severity: HIGH,CRITICAL
          exit-code: 1
          ignore-unfixed: true

      - name: Push Docker image
        if: success()

```

```
run: |  
    docker push ${ secrets.ACR_SERVER }/learningste  
ps:${ github.sha }
```

CD Code



CI/CD Workflows on GitHub

3.4 Kubernetes Deployment

- **Init container pattern:** the app pod refuses to start until the database schema exists — an idempotent SQL script (`CREATE TABLE IF NOT EXISTS ...`) mounted from a ConfigMap and executed by `psql` using the connection string from a Kubernetes secret. Idempotency matters because with 2 replicas, the second init container re-runs the script against tables the first already created (confirmed in logs: `NOTICE: relation "entries" already exists, skipping`).
- **Secrets flow:** Key Vault → (manual copy) → Kubernetes Secret → injected via `secretRef` . The connection string never exists in any file in the repository.

```
az aks get-credentials --resource-group <your_rg> --name <y  
our_aks>
```

AKS Credentials SHA tag

```
$KV    = "<your_kv_name>"  
$CONN = az keyvault secret show --vault-name $KV --name db-  
connection-string --query value -o tsv  
kubectl create secret generic learningsteps-secrets --from-  
literal=DATABASE_URL="$CONN"
```

Connection

```
image: <your_acr_server>/learningsteps:<full-SHA-AKS-tag>
```

Image pinning

- **Autoscaling:** HPA requires CPU `requests` in the deployment spec; without them, HPA targets show `<unknown>` and scaling never happens.
`resources.requests: cpu 100m` was set accordingly.
- **Declarative delivery:** the image reference in `deployment.yaml` is pinned to an exact commit SHA; updates are applied with `kubectl set image` or `kubectl apply`.

NAME	READY	STATUS	RESTARTS	AGE
learningsteps-9d68d4d55-8qgb4	1/1	Running	0	44s
learningsteps-9d68d4d55-dxw8v	1/1	Running	0	49s

Pods running via Kubectl get pods

```
PS C:\Users\Pupila\Documents\Cybersteps\Module 3\learningsteps-evolution> kubectl get hpa
NAME           REFERENCE                TARGETS  MINPODS  MAXPODS  REPLICAS  AGE
learningsteps  Deployment/learningsteps  cpu: 2%/70%  2        5        2         72m
PS C:\Users\Pupila\Documents\Cybersteps\Module 3\learningsteps-evolution>
```

hpa

4. Result

- LearningSteps API publicly reachable via LoadBalancer external IP (`http://<EXTERNAL-IP>/docs`), ports mapped 80 → 8000.
- Database reachable only from the cluster's private subnet: no public endpoint on the database.
- CRUD operations verified through the interactive API documentation.
- Full pipeline green: push → build → security gate → ACR → deployed pods.

LearningSteps API 0.1.0 OAS 3.1

/openapi.json

A simple learning journal API for tracking daily work, struggles, and intentions

default

GET	/entries	Get All Entries	✓
POST	/entries	Create Entry	✓
DELETE	/entries	Delete All Entries	✓
GET	/entries/{entry_id}	Get Entry	✓
PATCH	/entries/{entry_id}	Update Entry	✓
DELETE	/entries/{entry_id}	Delete Entry	✓

API from browser

Server response

Code	Details
200	Response body <pre>{ "entries": [{ "id": "56a29a91-5956-4a44-a9c8-bff54f380dea", "work": "Finishing past projects", "struggle": "Troubleshooting errors, revisiting past topics", "intention": "Keep on track", "created_at": "2026-10-03T00:04:22.116716+00:00", "updated_at": "2026-10-03T00:04:22.116716+00:00" }], "count": 1}</pre> Download

Testing server response

Assignment criterion	Status
API accessible via public IP	✓
CRUD via API endpoints	✓
Pipeline with security gates	✓ Trivy fails build on HIGH/CRITICAL
IaC provisioning	✓ full stack reproducible with <code>terraform apply</code>
Self-healing / scalable deployment	✓ Deployment + HPA (2-5 replicas)
Optional: Prometheus/Grafana	✗ Not implemented — documented as future work

5. Challenges & Root-Cause Analysis

This section documents five distinct failures and how each was resolved. The recurring methodology: **stop changing variables at random; read the actual**

error, form a hypothesis, and test it against evidence.

5.1 Pipeline failure

Symptom: CD pipeline failed at the Trivy step; no image pushed.

Misdiagnosis: initially perceived as a pipeline error.

Root cause: the gate was working exactly as configured → `exit-code: 1` with no severity filter fails on *any* finding, including LOW/MEDIUM noise in the base image.

```

147 ▼ [X] Scan image with Trivy 11s
148   ▶ Run entrypoint.sh
149   Running Trivy with options: trivy image ***/learningsteps:c19143fba8b9b2bca2f5d254e459dd3e59f7ea40
160   2026-10-02T19:54:53Z INFO [vulndb] Need to update DB
161   2026-10-02T19:54:53Z INFO [vulndb] Downloading vulnerability DB...
162   2026-10-02T19:54:53Z INFO [vulndb] Downloading artifact... repo="mirror.gcr.io/aquasec/trivy-db:2"
163   2026-10-02T19:54:53Z ERROR [vulndb] Failed to download artifact repo="mirror.gcr.io/aquasec/trivy-db:2"
err="oci download error: failed to fetch the layer: GET https://mirror.gcr.io/v2/aquasec/trivy-
db/blobs/sha256:dec4a29fa385849123ae98d60b0431c4e6a7da700aa75b7aa6c4146851ede89d: BLOB_UNKNOWN: Unknown blob:
sha256:dec4a29fa385849123ae98d60b0431c4e6a7da700aa75b7aa6c4146851ede89d"
164   2026-10-02T19:54:53Z INFO [vulndb] Trying to download artifact from other repository...
165   2026-10-02T19:54:53Z INFO [vulndb] Downloading artifact... repo="ghcr.io/aquasecurity/trivy-db:2"
```

Trivy scan fail

```

Error: Process completed with exit code 1.
▼ [✓] Remove Trivy Envs file
▶ Run rm -f trivy_envs.txt
```

exit-code: 1

```

Python (python-pkg)
=====
Total: 6 (UNKNOWN: 0, LOW: 0, MEDIUM: 2, HIGH: 4, CRITICAL: 0)
```

Vulnerabilities found

Fix: align the gate with the requirement ("fail on HIGH or CRITICAL"):

```

name: Scan image with Trivy
uses: aquasecurity/trivyaction@v0.36.0
with: image-ref: ${ secrets.ACR_SERVER }}/learningsteps:${ github.sha } severity:
HIGH,CRITICAL
exit-code: 1 ignore-unfixed: true

```

Lesson:

the first question when a control fires is whether the control is wrong or the signal is real. This is alert triage, and it's the core of SOC work.

5.2 Debian package CVE

Symptom: after the severity fix, the pipeline still failed: `libpcre2-8-0 10.46-1~deb13u2`, CVE-2026-103111 (HIGH, out-of-bounds write via crafted regex), fixed version available.

Root cause: stale OS package in the base image.

Fix:

```

RUN apt-get update && \
  apt-get upgrade -y --only-upgrade libpcre2-8-0 && \
  rm -rf /var/lib/apt/lists/*

```

(`--only-upgrade` was necessary because Debian's phased updates make a blanket `apt-get upgrade` skip packages that are being rolled out gradually.)

Related: three Python packages carried HIGH CVEs (msgpack, setuptools, urllib3).

Python (python-pkg)

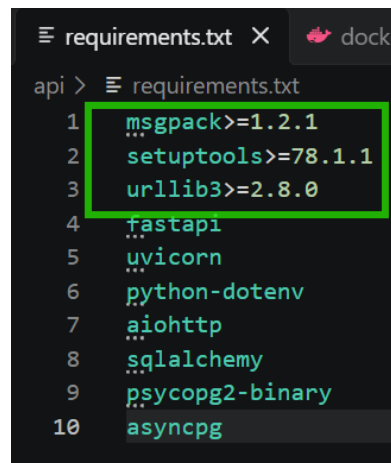
=====

Total: 6 (UNKNOWN: 0, LOW: 0, MEDIUM: 2, HIGH: 4, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
msgpack	GHSA-6v7p-g79w-8964	HIGH	fixed	1.1.2	1.2.1	MessagePack for Python: Out-of-bounds read / crash on Unpacker reuse after a... https://github.com/advisories/GHSA-6v7p-g79w-8964
setuptools	CVE-2025-47273			70.3.0	78.1.1	setuptools: Path Traversal Vulnerability in setuptools PackageIndex https://avd.aquasec.com/nvd/cve-2025-47273
	CVE-2026-59890	MEDIUM			83.0.0	setuptools: setuptools: MANIFEST.in exclusion bypass in sdist via Unicode normalization collision (NFC/NFD)... https://avd.aquasec.com/nvd/cve-2026-59890
urllib3	CVE-2026-97687	HIGH		2.7.0	2.8.0	urllib3: urllib3: Traffic interception via HTTPS proxy TLS configuration override https://avd.aquasec.com/nvd/cve-2026-97687
	CVE-2026-97689					urllib3: urllib3: Denial of Service via unbounded memory allocation in chunk parser... https://avd.aquasec.com/nvd/cve-2026-97689
	CVE-2026-97688	MEDIUM				urllib3: urllib3: Denial of Service via infinite loop during chunked Deflate decoding... https://avd.aquasec.com/nvd/cve-2026-97688

Local scan

Resolved by raising minimum versions in `requirements.txt` (`msgpack>=1.2.1`, `setuptools>=78.1.1`, `urllib3>=2.8.0`).



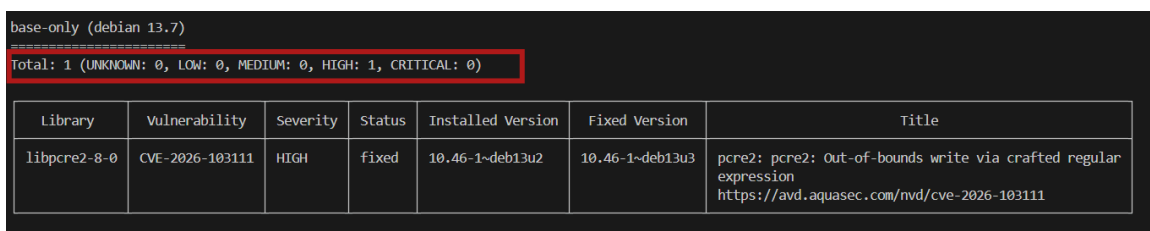
```
requirements.txt
api > requirements.txt
1 msgpack>=1.2.1
2 setuptools>=78.1.1
3 urllib3>=2.8.0
4 fastapi
5 uvicorn
6 python-dotenv
7 aiohttp
8 sqlalchemy
9 psycopg2-binary
10 asyncpg
```

Requirements file update

5.3 Scanner false positives

Symptom: after the fixes, Trivy *still* reported the old versions (msgpack 1.1.2, setuptools 70.3.0, urllib3 2.7.0) but the CD build log and `pip list` inside the image proved the patched versions were installed.

Investigation — bisection: scanning a bare `FROM python:3.13-slim` image (no application code) reproduced only the libpcre2 finding and **zero** Python vulnerabilities.



```
base-only (debian 13.7)
=====
Total: 1 (UNKNOWN: 0, LOW: 0, MEDIUM: 0, HIGH: 1, CRITICAL: 0)
```

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
libpcre2-8-0	CVE-2026-103111	HIGH	fixed	10.46-1-deb13u2	10.46-1-deb13u3	pcre2: pcre2: Out-of-bounds write via crafted regular expression https://avd.aquasec.com/nvd/cve-2026-103111

Base-only scan

The per-file scan showed every installed package clean; the aggregate `"Target": "Python"` section contradicted all of it.

Root cause: the flagged versions existed nowhere on the image filesystem the findings originated from pip's internal vendor libraries in the base image, which cannot be patched through `requirements.txt` at all.

Fix: documented suppression in `.trivyignore` file:

```

# .trivyignore
1 # trivyignore - suppressing validated false positives
2 # Source: pip's internal vendored libraries in python:3.13-slim base image
3 # Verification: pip list in container shows patched versions; base-only scan confirms no user-space
4 GHSA-6v7p-g79w-8964 # msgpack vendored in pip/_vendor (installed 1.2.3)
5 CVE-2025-47273 # setuptools vendored in pip/_vendor (installed 84.0.0)
6 CVE-2026-97687 # urllib3 vendored in pip/_vendor (installed 2.8.0)
7 CVE-2026-97689 # urllib3 vendored in pip/_vendor (installed 2.8.0)

```

.trivyignore file

5.4 Hollow image

Symptom: pods failed with `exec: "uvicorn": executable file not found in $PATH`.

Investigation: pulling the deployed image from ACR and running `pip list` inside it showed exactly one package: pip itself. During Dockerfile edits for §5.2, the `COPY api/requirements.txt` / `RUN pip install` layers had been lost. The CMD survived, pointing at a binary that no longer existed.

Why the pipeline didn't catch it: a stripped image has no Python packages to scan. Trivy reported zero vulnerabilities, the gate passed, and a broken artifact shipped.

Fix: restored Dockerfile, added a mandatory pre-push verification step:

```

docker build -t ls-test .
docker run --rm --entrypoint pip ls-test list # must show full dependency list

```

5.5 Connection-string parsing — the @ in the password

Symptom: init container in `CrashLoopBackOff`.

NAME	READY	STATUS	RESTARTS	AGE
learningsteps-5c8f5cf95d-dspz8	0/1	Init:CrashLoopBackOff	8 (2m44s ago)	18m
learningsteps-7d45f94b7c-q9gnk	0/1	Init:Error	9 (5m21s ago)	21m
learningsteps-7d45f94b7c-t5x8h	0/1	Init:Error	9 (5m8s ago)	21m
learningsteps-7d45f94b7c-q9gnk	0/1	Init:CrashLoopBackOff	9 (72s ago)	22m
learningsteps-7d45f94b7c-t5x8h	0/1	Init:CrashLoopBackOff	9 (72s ago)	22m
learningsteps-5c8f5cf95d-dspz8	0/1	Init:Error	9 (5m4s ago)	21m
learningsteps-5c8f5cf95d-dspz8	0/1	Init:CrashLoopBackOff	9 (66s ago)	22m

Troubleshooting `CrashLoopBackOff`

Investigation via logs:

```

kubectl logs <pod>
psql: error: could not translate host name "ssw0rd24@learningsteps...-
pg.postgres.database.azure.com" to address: Name does not resolve.

```

Root cause: the database password contained @, the reserved userinfo/host delimiter in a URL. psql parsed the second half of the password as the

hostname.

Fix: URL-encode reserved characters in the password (`@` → `%40`) when constructing the connection string, then recreate the secret and roll the deployment:

- `kubectl delete secret learningsteps-secrets`
- `kubectl create secret generic learningsteps-secrets --from literal=DATABASE_URL="
<encoded-url>"`
- `kubectl rollout restart deployment/learningsteps`

6. Key Learnings

1. **Alert triage is a transferable skill.** Four of the five failures were resolved by asking "is the detector wrong, or is the signal real?" before changing anything; the same discipline as SOC alert analysis.
2. **Green pipelines can lie.** The hollow image shipped *because* it scanned clean. Verification must target the artifact (running `pip list` in the image), not the pipeline status.
3. **Scan findings need a disposition, not just a fix.** Suppressed-with-justification (`.trivyignore`), fixed (`-only-upgrade` , version floors), or accepted (documented) — each finding got a deliberate treatment, which is what "DevSecOps" means in practice.
4. **Kubernetes' declarative model cuts both ways.** Pods never fix their own spec; files never apply themselves.
5. **Idempotent initialization enables replication.** The `IF NOT EXISTS` schema script is why two replicas can race to initialize the same database safely.

7. Future Work

- **Observability:** Prometheus + Grafana via the `kube-prometheus-stack` Helm chart; app instrumentation with `prometheus-fastapi-instrumentator` ; dashboards for request volume, p95 latency, and error rates.
- **Pipeline hardening:** the CI lint/test steps currently swallow failures (`|| true`); with tests present, they should become real gates. The final deploy stage (auto-`kubectl set image` after push) is manual — scripting it closes the loop.

- **Secrets automation:** replace the manual Key Vault → Kubernetes secret copy with a CSI driver or workload identity federation.
- **Multi-stage Docker build:** reduce attack surface by excluding pip and its vendor tree from the runtime image, which would also eliminate the §5.3 false-positive class structurally.